

IMPLEMENTATION GENERALIZATION PLAN

Agentic Generative AI Framework for Autonomous Software Testing, Failure Investigation, Root-Cause Analysis, and Regression Test Generation

Purpose: evolve the existing login-focused proof of concept into a requirement-driven, SUT-aware academic prototype without rebuilding the working architecture.

1. Non-Negotiable Goal

The framework must not depend on a fixed list of input sentences, keywords, or login-only requirements. It must accept arbitrary natural-language software requirements and attempt to interpret them dynamically. The system should determine whether enough target-application context exists to create executable tests. When the requirement is ambiguous, unsupported, non-testable, or lacks necessary SUT information, it must return a structured clarification/unsupported result instead of silently mapping the input to a hardcoded login flow.

Important engineering boundary: accepting any input text does not mean every sentence can always be executed as a valid software test. The framework should robustly handle any text input, classify/interpret it, and either generate an appropriate testing plan or clearly explain what additional information is required.

2. What Must Be Preserved

- FastAPI backend and existing API structure where possible.
- LangGraph orchestration and shared workflow state.
- LangChain/LLM integration.
- Selenium + PyTest execution engine.
- Multi-test execution and execution summaries.
- Failure evidence, screenshots, exceptions, stdout/stderr and existing execution artifacts.
- RAG layer and project knowledge base.
- Failure Investigation / Root-Cause Analysis.
- Verification Agent.
- Bug Report Agent.
- Regression Test Agent and regression execution.
- Human-in-the-loop approval flow.
- PostgreSQL persistence and Alembic migrations.
- n8n orchestration and existing run/approve API compatibility.
- React frontend and existing result sections.

3. What Must Be Removed or Replaced

Remove / Replace	Required change
Login-only requirement rejection	Remove logic that treats requirements as valid only when they refer to the demo login application.

Fixed login scenario lists	Do not hardcode valid-login, invalid-password, invalid-username, empty-username and empty-password as the universal scenario set.
Login-specific automation assumptions	Remove general workflow assumptions about /login, username/password fields, login_button, dashboard title, or securepass/testuser.
Login-specific expected-result mapping	Assertions must be generated from the structured test case and SUT context, not from login keywords.
Login-specific RCA conclusions	RCA must reason from the actual failed test and evidence. SUT-specific knowledge may be supplied as context, but must not be embedded as universal logic.
Fixed-input routing	Do not create a whitelist of accepted sentences or keyword-to-test mappings. Similar requirements phrased differently must still be interpretable.

4. What Must Be Added

Add	Purpose
Generic request model / SUT context	Accept requirement text plus target application context such as base URL, optional feature/page context, credentials/test data references, and available SUT metadata. Keep backward compatibility where practical.
Requirement Understanding Agent	Use the LLM to transform arbitrary natural-language input into structured intent: feature, objective, actors, inputs, preconditions, expected behavior, constraints, ambiguities and testability.
Input classification and clarification	Classify input as testable requirement, ambiguous/incomplete requirement, unsupported/non-software request, or unsafe/unexecutable request. Never crash or force every sentence into a login test.
Dynamic Scenario Generation	Generate applicable functional, negative, boundary and security-oriented scenarios from the interpreted requirement. Security scenarios should be generated only when relevant.
Dynamic Structured Test Cases	Create test IDs, type, title, preconditions, steps, test data, expected result and traceability back to the requirement/scenario.
SUT Discovery / SUT Context Layer	Provide the Automation Agent with real target-application information: base URL, pages/routes, element selectors or accessible page structure, and known test data. Avoid guessing selectors.
Generic Automation Agent	Generate Selenium/PyTest automation from the structured test case + SUT context. The same generation path must work for login and at least one non-login feature.
Structured evidence package	On failure, collect available URL/title/page source,

	screenshot, exception/stack trace, stdout/stderr and browser console logs where practical. Keep evidence collectors extensible.
Traceability	Maintain requirement -> scenario -> test case -> generated code -> execution result -> failure analysis -> bug report -> regression test relationships.
Multi-Agent Activity UI	Show Requirement, Test Generation, Automation, Investigation, Verification, Bug Report and Regression agents plus non-agent workflow components. Statuses must be derived from real API data: Completed, Waiting, Failed, Skipped.

5. Target Workflow

User Input + Target SUT Context

- > Requirement Understanding Agent
- > Testability / Clarification Decision
- > Dynamic Scenario Generation
- > Structured Test-Case Generation
- > RAG / Project Context Retrieval where relevant
- > Automation Agent
- > Selenium/PyTest Execution
- > Execution Summary
- > If PASS: persist and finish
- > If FAIL: Evidence Collection
- > Investigation / Root-Cause Analysis
- > Verification Agent
- > Bug Report Agent
- > Regression Test Agent
- > Human Approval
- > Regression Execution
- > Persist final state in PostgreSQL

n8n remains the external orchestration/HITL layer; React remains the demonstration dashboard.

6. Detailed Implementation Phases

Phase 0 - Baseline and Freeze

- Create a Git checkpoint before generalization.
- Record one known-good login PASS result and one known-good failure-path result.
- Do not redesign PostgreSQL, n8n, HITL, regression execution, or the frontend API contract unless a required field must be added.

Phase 1 - Generic Input Contract

- Define a generic workflow input containing at minimum requirement text and target base URL/SUT identifier.
- Allow optional SUT context/test data without making them mandatory for every input.

- Ensure arbitrary text never causes an unhandled exception.
- Preserve existing API fields needed by n8n/frontend.

Phase 2 - Requirement Understanding Agent

- Replace login-only validation with LLM-based structured interpretation.
- Produce structured fields such as feature, objective, actors, inputs, preconditions, expected behavior, constraints, ambiguities and testability.
- For incomplete requirements, return `clarification_needed` with a reason rather than inventing missing facts.
- For unrelated text, return `unsupported/not_testable` rather than generating fake tests.
- Test paraphrases so behavior is semantic, not keyword-dependent.

Phase 3 - Dynamic Scenario Generation

- Generate scenarios from requirement understanding rather than a fixed list.
- Support functional, negative, boundary and security-oriented categories when applicable.
- Do not require exactly five scenarios; count should depend on the requirement.
- Store scenario IDs/types/descriptions in workflow state.

Phase 4 - Dynamic Test-Case Generation

- Generate structured cases from each scenario.
- Include `test_id`, `scenario_id`, `test_type`, `title`, `preconditions`, `steps`, `test data` and `expected result`.
- Validate LLM structured output before continuing.
- Maintain traceability to the original requirement.

Phase 5 - SUT Context / Discovery

- Separate the framework from `demo_app`-specific knowledge.
- Introduce a SUT context abstraction containing base URL and available page/element metadata.
- For the academic prototype, SUT metadata may come from a configuration/manifest or controlled discovery rather than attempting universal autonomous web crawling.
- The Automation Agent must not invent selectors when required information is unavailable; it should request/return missing context.

Phase 6 - Generic Automation Agent

- Generate Selenium/PyTest code from test case + SUT context.
- Remove universal references to login URL, username, password, dashboard and `login_button`.
- Use the same generator for all supported UI features.
- Retain syntax/quality safeguards and controlled execution.
- Keep Selenium/PyTest as the implemented prototype executor; describe Playwright/API/TestNG as extension points unless actually implemented.

Phase 7 - Execution and Evidence

- Continue multi-test execution and aggregate total/passed/failed.
- Map every execution result back to its `test_id`.
- On failure, preserve screenshot, exception, stack trace/stdout/stderr and generated code.
- Add current URL, page title, page source/HTML and browser console logs where practical.

- Do not claim network/API evidence unless it is actually captured.

Phase 8 - Generic Investigation, RAG and Verification

- Build the investigation prompt/context from requirement, failed test, code, evidence and retrieved knowledge.
- Remove login-specific RCA rules except SUT-specific knowledge explicitly supplied as context.
- RAG should retrieve project-specific documents relevant to the failed feature/task.
- Verification must check whether the proposed root cause is supported by evidence and expose uncertainty/confidence.

Phase 9 - Bug Report and Regression

- Generate the bug report from the actual failed test and verified investigation.
- Generate a regression test tied to the defect/root cause and original requirement.
- Preserve HITL approval before regression execution.
- Persist approval and regression result in PostgreSQL.

Phase 10 - Second Non-Login SUT Feature

- Extend the controlled demo SUT with at least one clearly different feature, e.g. profile display-name update or simple product/cart behavior.
- Do not add special-case workflow code for that feature.
- Run the same generic pipeline against login and the second feature.
- This is the key proof that the framework is requirement-driven rather than a login tester.

Phase 11 - Multi-Agent Frontend

- Add a Multi-Agent Workflow/Activity section.
- Show real agent/component status based only on API data.
- On a PASS path, failure-only agents/components must show Skipped.
- On a failure path, show Investigation -> RAG -> Verification -> Bug Report -> Regression -> Human Review -> Regression Execution.
- Keep existing detailed sections for scenarios, cases, code, evidence and results.

Phase 12 - n8n Compatibility Verification

- Do not move agent logic into n8n.
- Keep n8n as orchestration/HITL integration.
- Verify existing Run Autonomous Test and Approve/Reject flows still work with the generalized API.
- Only update request payload mapping if the generic input contract requires additional fields.

Phase 13 - Final Demonstration and Report Alignment

- Demo at least two semantically different requirements/features.
- Show one all-PASS workflow.
- Show one controlled failure with evidence, RCA, verification, bug report, regression generation, approval and regression execution.
- Capture PostgreSQL persistence and n8n orchestration evidence.
- Update report wording so implemented capabilities are distinguished from extensible/future capabilities.

7. Required Behavior for Arbitrary Input Text

This is the central acceptance rule. The application must be input-robust and semantics-driven. It must not be built around a small collection of exact sentences. For every submitted text, the workflow must produce a controlled outcome.

Input situation	Expected framework behavior
Clear testable software requirement	Interpret -> generate scenarios/cases -> automate if sufficient SUT context exists.
Same requirement expressed with different wording	Produce semantically equivalent understanding without requiring exact keywords.
Incomplete requirement	Return clarification_needed and identify missing information; do not invent critical facts.
Ambiguous requirement	Expose ambiguity and request/record clarification.
Non-software sentence	Return unsupported/not_testable cleanly; do not map it to login.
Valid requirement but missing SUT selectors/context	Generate planning/test cases if possible, but stop executable automation with a structured missing_context result.
Unsupported testing modality	Return unsupported_executor/capability rather than fake execution.
Malformed/empty/very long text	Validate safely and return a structured error/clarification result without crashing.

8. Acceptance Criteria

- No universal workflow branch checks whether the requirement is specifically about login.
- No fixed list of accepted natural-language sentences.
- No exact-keyword dependency for normal requirement interpretation.
- Login remains a valid test case but is only one SUT feature.
- At least one non-login feature runs through the same Requirement -> Scenario -> Test Case -> Automation -> Execution pipeline.
- Scenario count and content are dynamically generated.
- Generated automation uses SUT context rather than globally hardcoded selectors.
- Failures are analyzed using the actual failed test and collected evidence.
- Unsupported/ambiguous inputs produce structured outcomes, not hallucinated tests or crashes.
- PASS and FAIL paths remain persisted in PostgreSQL.
- n8n run and approval flow remains functional.
- Frontend accurately displays multi-agent/component activity and detailed artifacts.
- Report claims match what is demonstrably implemented.

9. Testing Matrix Before Code Freeze

Test	Input/Condition	Expected
A	Login requirement - normal wording	Dynamic scenarios/cases; execution works.
B	Login requirement - paraphrased wording	Same intent understood without exact sentence matching.

C	Non-login feature requirement	Same generic pipeline works; no login assumptions.
D	Non-login requirement - paraphrased	Semantic interpretation remains correct.
E	Ambiguous requirement	clarification_needed.
F	Unrelated sentence	unsupported/not_testable.
G	Missing SUT context	Structured missing_context; no invented selectors.
H	Controlled test failure	Evidence -> RCA -> verification -> bug -> regression -> HITL.
I	Approved regression after fix	Regression executes and result persists.
J	n8n flow	Run + decision + approval/rejection remain compatible.

10. Estimated Focused Work

Target approximately 10-16 focused hours for a convincing generalized academic prototype, assuming the existing working architecture is preserved. Do not spend time implementing every possible browser, framework, network collector, or arbitrary external website. The priority is semantic requirement handling, generic scenario/test generation, SUT-aware automation, a second non-login feature, and end-to-end evidence.

11. Work Discipline for the New Chat / Coding Agent

- Read this plan first and treat it as the implementation contract.
- Before editing, inspect only the minimum files needed for the current phase.
- Work one phase/small task at a time; do not perform a broad repository rewrite.
- Preserve working architecture and backward compatibility.
- Do not remove PostgreSQL, n8n, LangGraph, LangChain, RAG, HITL, regression execution or frontend functionality.
- Do not replace dynamic behavior with a larger set of hardcoded inputs.
- After each phase, report files changed, behavior added, tests/manual verification performed, and remaining limitations.
- Do not claim generalization until both login and a non-login feature pass through the same generic pipeline.
- Stop and explain before making a large architectural change that contradicts this plan.

12. Final Deliverable Definition

The final project should be presented as a requirement-driven agentic testing prototype. A user submits arbitrary natural-language text plus sufficient target-application context. The framework interprets the input, determines testability, generates relevant scenarios and structured test cases, creates and executes automation where supported, analyzes real failures using evidence and RAG, verifies the analysis, generates a defect and regression test, supports human approval, executes regression, persists the

workflow, and exposes the process through the dashboard and n8n orchestration. The demo login application is one reference SUT, not the definition of the framework.